

FutureKawa

Documentation Backend — Siège

Équipe : Rudy · Florian · Noé · Josselin

Version : 1.0

Date : 05/07/2026

Projet : FutureKawa

Sommaire

Backend Siège — FutureKawa

01 — Vue d'ensemble

02 — Django & Django REST Framework

03 — Endpoints de l'API

04 — Agrégation des exploitations

05 — Sécurité & chiffrement

06 — Configuration & déploiement

07 — Tests

Backend Siège — FutureKawa

Projet	FutureKawa
Livrable	Documentation Backend Siège
Périmètre	API Django centrale qui agrège les plantations
Framework	Django 4.2 + Django REST Framework
Communication	Client HTTP (requests) vers les exploitations
Sécurité	Tokens API chiffrés (Fernet / cryptography)
Équipe	Rudy · Florian · Noé · Josselin

Sommaire

Section	Fichier
Vue d'ensemble	01-vue-ensemble.md
Django & Django REST Framework	02-django-drf.md
Endpoints de l'API	03-endpoints-api.md
Agrégation des exploitations	04-agregation-sites.md
Sécurité & chiffrement	05-securite-chiffrement.md
Configuration & déploiement	06-configuration.md
Tests	07-tests.md

En quoi le siège diffère de l'exploitation

	Siège	Exploitation
Base de données	Centrale (sites + utilisateurs)	Locale (données terrain)

	Siège	Exploitation
Temps réel MQTT	Non	Oui
Celery / Redis	Non	Oui
Rôle réseau	Client : consomme les exploitations	Serveur : expose <code>/api/all-data/</code>
Chiffrement	Tokens API des sites (Fernet)	—

01 — Vue d'ensemble

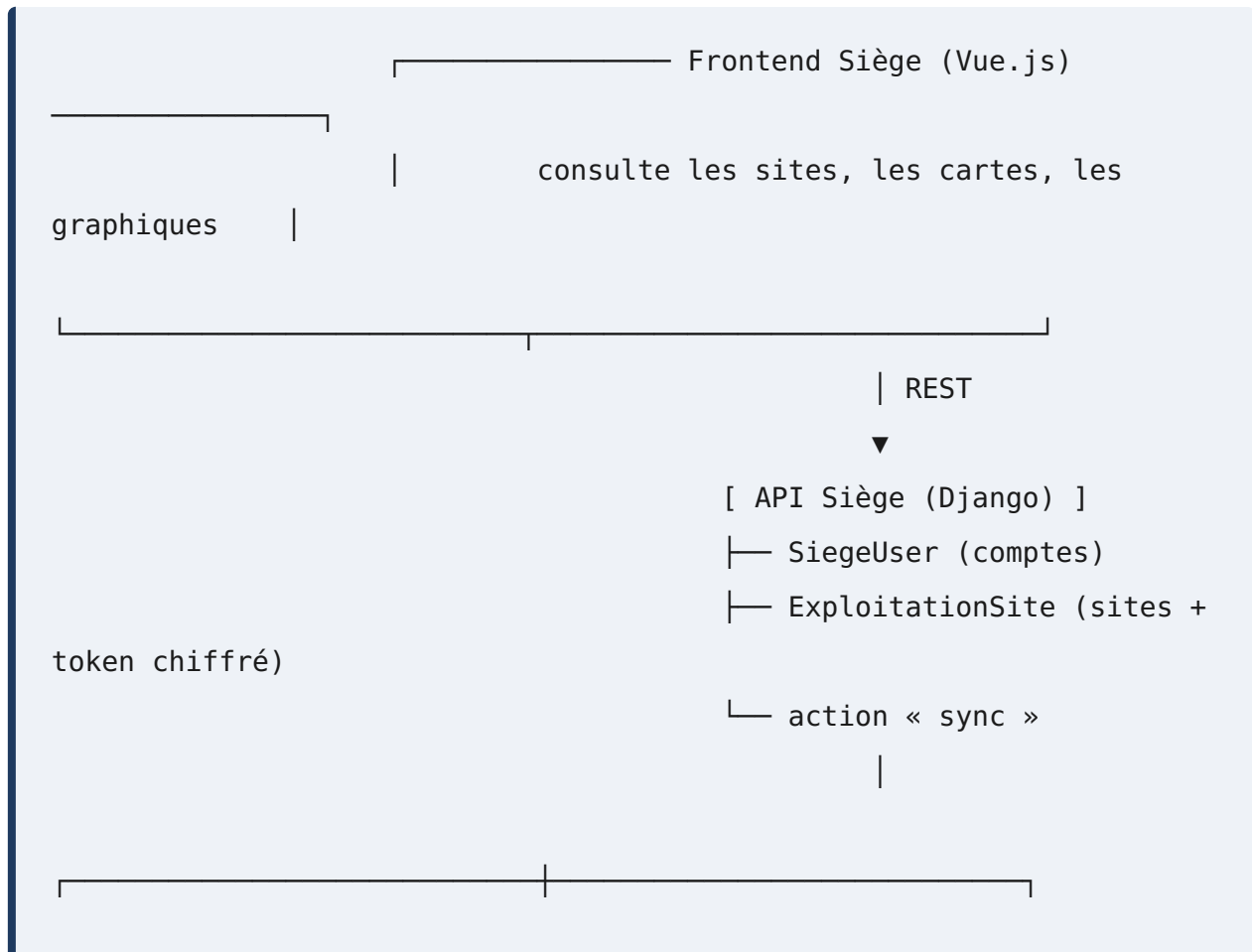
Rôle du backend siège

Le backend siège est la **tour de contrôle multi-plantations**. Il ne collecte pas lui-même de mesures : il **interroge** chaque exploitation, en cache les données, et offre une vue consolidée au frontend siège.

Ses missions :

1. **Référencer** les sites d'exploitation (nom, pays, URL d'API, token d'accès chiffré).
2. **Synchroniser** : appeler l'endpoint `/api/all-data/` de chaque exploitation.
3. **Cacher** la dernière réponse de chaque site (`last_data` , `last_sync`).
4. **Gérer** les utilisateurs du siège (admin / lecteur) et leur authentification.

Architecture réseau



```

| GET /api/all-data/ | GET /api/all-
data/ | ...
      ▼ (Token chiffré)
▼
      [ Exploitation A ]      [ Exploitation B ]
[ Exploitation C ]

```

Le siège est **client** des exploitations. Il n'accède jamais à leurs bases de données : il passe par leur API, authentifié avec un token propre à chaque site.

Stack technique

Élément	Choix	Rôle
Framework web	Django 4.2	Structure, ORM, admin
API	Django REST Framework	Sérialisation, ViewSets
Base de données	PostgreSQL 15	Sites + utilisateurs
Client HTTP	<code>requests</code>	Appels vers les exploitations
Chiffrement	<code>cryptography</code> (Fernet)	Tokens API des sites
Filtres	<code>django-filter</code>	Filtrage des listes API
Doc API	<code>drf-spectacular</code>	Swagger / OpenAPI

Dépendances (`requirements.txt`)

```

Django>=4.2,<5.0
djangorestframework>=3.14
psycpg2-binary>=2.9
django-cors-headers>=4.0
python-dotenv>=1.0.0
django-filter>=23.5      # filtrage des listes
drf-spectacular>=0.26.0  # documentation OpenAPI

```

```
requests>=2.31.0      # appels HTTP vers les exploitations
cryptography>=42.0    # chiffrement Fernet des tokens
```

Différence marquante avec l'exploitation : **pas de Celery, pas de Redis, pas de paho-mqtt**. En revanche, *requests* et *cryptography* sont propres au siège.

Les trois modèles

Modèle	Rôle
SiegeUser	Utilisateur du siège (<code>AbstractUser</code>), rôle <code>admin</code> ou <code>viewer</code>
ExploitationSite	Un site distant : URL d'API + token chiffré + cache des données
InvitationToken	Token UUID à usage unique pour définir un mot de passe (24 h)

02 — Django & Django REST Framework

Rappel : Django & DRF

Le siège partage le même socle que l'exploitation : **Django** (framework web Python, ORM, migrations) et **Django REST Framework** (API JSON). Le fonctionnement général — modèles, serializers, viewsets, routeurs — est décrit en détail dans le [livrable Backend Exploitation](#). Cette section se concentre sur ce qui est **propre au siège**.

Particularité 1 : un modèle utilisateur personnalisé

Contrairement à l'exploitation qui utilise le `User` standard de Django, le siège définit **son propre modèle utilisateur** en héritant d' `AbstractUser` :

```
# siege/models.py
class SiegeUser(AbstractUser):
    ROLE_CHOICES = [
        ('admin', 'Administrateur'),
        ('viewer', 'Lecteur'),
    ]
    role = models.CharField(max_length=20, choices=ROLE_CHOICES,
default='viewer')
    email = models.EmailField(unique=True)
```

Ajout	Raison
Champ <code>role</code>	Distinguer administrateurs et simples lecteurs
<code>email</code> unique	L'email sert d'identifiant fort (invitations)

Django est configuré avec `AUTH_USER_MODEL = 'siege.SiegeUser'` . C'est une décision qui se prend **dès le début** d'un projet, car changer de modèle utilisateur ensuite est coûteux.

Particularité 2 : un champ de modèle sur mesure

Le siège crée un **type de champ personnalisé**, `EncryptedCharField` , qui chiffre/déchiffre automatiquement sa valeur à la traversée de la base :

```
class EncryptedCharField(models.TextField):
    def from_db_value(self, value, expression, connection):
        return decrypt_token(value) if value else value    # lecture →
déchiffre

    def get_prep_value(self, value):
        return encrypt_token(value) if value else value    # écriture
→ chiffre
```

Utilisé pour le token d'accès aux exploitations :

```
class ExploitationSite(models.Model):
    api_token = EncryptedCharField()    # jamais stocké en clair
```

Le code applicatif manipule le token en clair ; le chiffrement est **transparent**, géré par l'ORM. Détails dans [Sécurité & chiffrement](#).

Particularité 3 : `django-filter`

Le siège ajoute `django_filters` pour filtrer les listes de l'API sans écrire de code :

```
# ex. filtrer les sites par pays ou statut
GET /api/sites/?country=Colombie&status=active
```

Authentification par token

Le siège active `rest_framework.authtoken` : à la connexion, l'utilisateur reçoit un token porté ensuite dans chaque requête.

```
# à la connexion
token, _ = Token.objects.get_or_create(user=user)
# → { "token": "..."}

# dans les requêtes suivantes
Authorization: Token <clé>
```

Un **throttle** limite les tentatives de connexion (`LoginRateThrottle`) pour contrer le brute-force.

Documentation automatique

Comme l'exploitation, le siège expose sa doc OpenAPI via `drf-spectacular` :

URL	Contenu
<code>/api/schema/</code>	Schéma OpenAPI
<code>/api/docs/</code>	Swagger interactif
<code>/api/redoc/</code>	ReDoc

Interface d'administration

Le siège active `django.contrib.admin` (l'exploitation ne l'utilise pas). L'admin Django offre une interface auto-générée pour gérer sites et utilisateurs, accessible sur `/admin/`.

03 — Endpoints de l'API

Point d'entrée

Toutes les routes sont sous `/api/` (`core/urls.py` → `siege.api.urls`), plus l'admin Django sur `/admin/`.

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('siege.api.urls')),
    path('api/schema/', SpectacularAPIView.as_view()),
    path('api/docs/', SpectacularSwaggerView.as_view()),
    path('api/redoc/', SpectacularRedocView.as_view()),
]
```

Ressources CRUD

Route	ViewSet	Ressource
<code>/api/sites/</code>	<code>ExploitationSiteViewSet</code> et	Sites d'exploitation
<code>/api/users/</code>	<code>SiegeUserViewSet</code>	Utilisateurs du siège

Endpoints d'authentification

Route	Méthode	Rôle
<code>/api/auth/login/</code>	POST	Connexion → renvoie un token
<code>/api/auth/logout/</code>	POST	Déconnexion → supprime le token
<code>/api/auth/set-password/</code>	POST	Définition du mot de passe via token d'invitation
<code>/api/health/</code>	GET	Sonde de santé

Route	Méthode	Rôle
/api/hello/	GET	Test de connectivité

Actions personnalisées sur les sites

Le `ExploitationSiteViewSet` ajoute des actions au-delà du CRUD standard : ce sont elles qui font du siège un **proxy** vers les exploitations.

Action	Route	Rôle
sync	POST /api/sites/{id}/sync/	Appelle /api/all-data/ du site et met à jour le cache
lot_history	GET /api/sites/{id}/lots/{batch_id}/history/	Proxy vers l'historique d'un lot du site
sensor_history	GET /api/sites/{id}/sensors/{sensor_id}/history/	Proxy vers l'historique d'un capteur du site

Détail du mécanisme de synchronisation dans [Agrégation des exploitations](#).

Actions personnalisées sur les utilisateurs

Action	Route	Rôle
create	POST /api/users/	Crée l'utilisateur et envoie un email d'invitation
resend_invitation	POST /api/users/{id}/resend-invitation/	Régénère un token d'invitation (24 h)
destroy	DELETE /api/users/{id}/	Supprime l'utilisateur

Exemple : création d'un site

```
POST /api/sites/  
{  
  "name": "Plantation Andes",  
  "country": "Colombie",  
  "api_url": "https://exploit-andes.futurekawa.com",  
  "api_token": "9a8b7c6d..." // sera chiffré avant stockage  
}
```

Réponse (le token n'est jamais renvoyé en clair) :

```
{  
  "id": 4,  
  "name": "Plantation Andes",  
  "country": "Colombie",  
  "status": "unknown",  
  "last_sync": null  
}
```

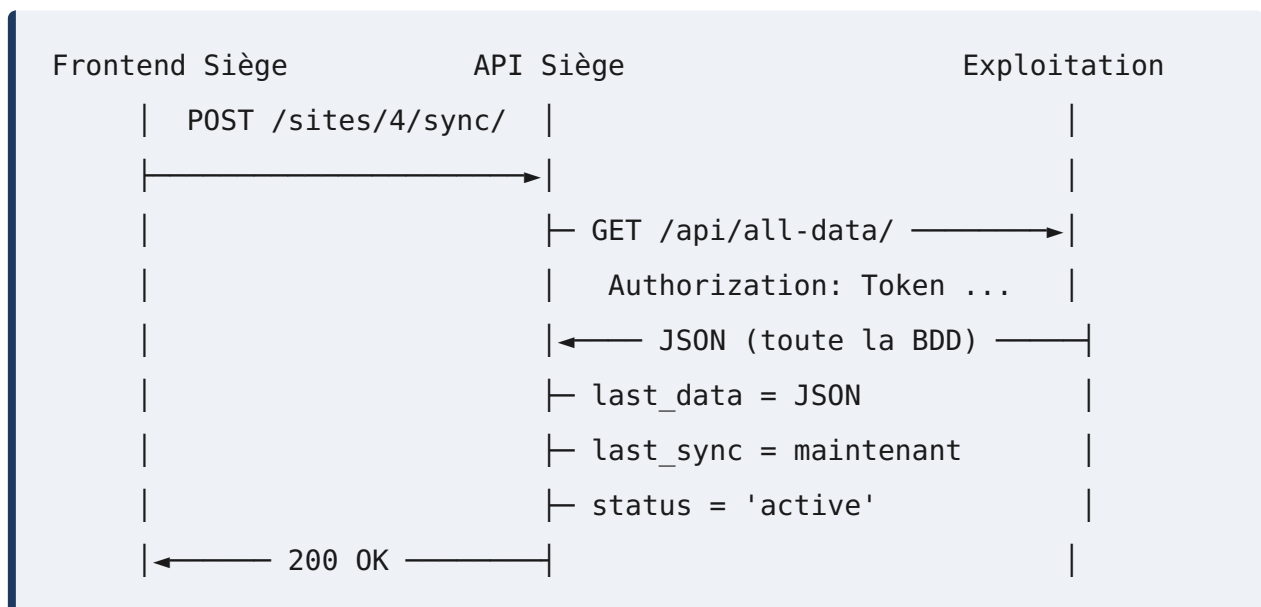
04 — Agrégation des exploitations

Le principe : un proxy qui met en cache

Le siège ne réplique pas les bases des exploitations. Il fait deux choses :

1. Il **appelle** l'endpoint `/api/all-data/` de chaque exploitation (avec le token du site).
2. Il **stocke** la réponse dans le champ `last_data` (JSON) du site, avec la date `last_sync`.

Le frontend siège lit ensuite ce cache — rapide, et disponible même si l'exploitation est momentanément injoignable.



L'action `sync` en détail

```
@action(detail=True, methods=['post'])
def sync(self, request, pk=None):
    site = self.get_object()
    url = site.api_url.rstrip('/') + '/api/all-data/'
    resp = http_requests.get(
        url,
        headers={'Authorization': f'Token {site.api_token}'}, #
        # déchiffré à la volée
        timeout=...,
```

```
)  
site.last_data = resp.json()  
site.last_sync = timezone.now()  
site.status = 'active'  
site.save()
```

`site.api_token` est **déchiffré automatiquement** par l'ORM au moment de la lecture (voir [Sécurité & chiffrement](#)). Le code n'a jamais à gérer le chiffrement manuellement.

Le statut de connexion

Chaque site porte un `status` qui reflète le résultat de la dernière synchronisation :

Statut	Signification
<code>unknown</code>	Jamais synchronisé
<code>active</code>	Dernière sync réussie
<code>error</code>	L'API a répondu une erreur (token invalide, 4xx/5xx)
<code>unreachable</code>	Site injoignable (timeout, DNS, réseau)

Cela permet au frontend d'afficher l'état de santé de chaque plantation d'un coup d'œil.

Le cache `last_data`

```
last_data = models.JSONField(null=True, blank=True) # toute la  
réponse /api/all-data/  
last_sync = models.DateTimeField(null=True, blank=True)
```

Avantages du cache :

Avantage	Explication
Rapidité	Le front lit du JSON local, pas 10 appels réseau distants
Résilience	Les données restent consultables si l'exploitation est hors ligne
Découplage	Le siège ne dépend pas de la disponibilité instantanée des sites

Proxys d'historique

Pour les données volumineuses (historiques de mesures/lots), le siège **ne cache pas** mais relaie la requête en direct vers l'exploitation :

```
GET /api/sites/4/sensors/12/history/
    |
    └─ GET {api_url}/api/sensors/12/history/ (Token du site)
```

Cela évite de dupliquer des milliers de mesures dans le cache du siège.

Résumé du flux inter-service

```
[Exploitation] expose /api/all-data/ (protégé par master token)
    ▲
    | requests.get(headers=Token)
    |
[Siège] action sync → last_data (cache JSON) → Frontend Siège
```


05 — Sécurité & chiffrement

Le risque : des tokens d'accès en clair

Le siège stocke, pour chaque exploitation, un **token d'accès** qui donne un droit de lecture complet sur les données de la plantation (`/api/all-data/`). Si la base du siège fuitait avec ces tokens **en clair**, un attaquant pourrait interroger directement toutes les exploitations.

La parade : **chiffrer les tokens au repos** (encryption at rest) avec **Fernet** (bibliothèque `cryptography`).

Fernet en bref

Fernet = chiffrement symétrique authentifié. Une même clé secrète chiffre et déchiffre ; le message est aussi signé, donc toute altération est détectée.

```
# siege/utils/encryption.py
from cryptography.fernet import Fernet

def _fernet() -> Fernet:
    return Fernet(settings.SITE_TOKEN_ENCRYPTION_KEY.encode())

def encrypt_token(plaintext: str) -> str:
    return _fernet().encrypt(plaintext.encode()).decode()

def decrypt_token(ciphertext: str) -> str:
    return _fernet().decrypt(ciphertext.encode()).decode()
```

La clé provient d'une variable d'environnement, **jamais du code** :

```
# core/settings/base.py
SITE_TOKEN_ENCRYPTION_KEY =
os.environ.get('SITE_TOKEN_ENCRYPTION_KEY', ...)
```

Chiffrement transparent via l'ORM

Le token n'est jamais chiffré/déchiffré « à la main » dans les vues. Le champ personnalisé `EncryptedCharField` s'en charge à chaque aller-retour base de données :

```
class EncryptedCharField(models.TextField):
    def from_db_value(self, value, expression, connection):
        return decrypt_token(value) if value else value    # lecture →
clair

    def get_prep_value(self, value):
        return encrypt_token(value) if value else value    # écriture
→ chiffré
```

Code applicatif		Base PostgreSQL
site.api_token = "abc"	—get_prep_value→	"gAAAAAB...=" (chiffré)
site.api_token	←from_db_value—	"gAAAAAB...=" → "abc"

Résultat : en base, la colonne `api_token` ne contient **que du chiffré** ; le reste du code voit toujours le token en clair.

Autres mesures de sécurité

Mesure	Mise en œuvre
Auth par token	<code>rest_framework.authtoken</code> — token émis à la connexion
Anti brute-force	<code>LoginRateThrottle</code> limite les tentatives de login
Tokens d'invitation	UUID à usage unique, expiration 24 h , supprimé après usage

Mesure	Mise en œuvre
Le token de site n'est jamais renvoyé	Absent des réponses de l'API (write-only)
CORS restreint en prod	<code>CORS_ALLOWED_ORIGINS</code> explicite (pas d' <code>ALLOW_ALL</code>)
Rôles	<code>admin</code> vs <code>viewer</code> — les lecteurs n'ont pas les droits d'écriture

Cycle d'un token d'invitation

```

Admin crée un utilisateur
|
├─ InvitationToken (UUID, expire dans 24 h)
├─ email envoyé : .../set-password?token=<uuid>
|
Utilisateur ouvre le lien → POST /api/auth/set-password/
├─ token valide et non expiré ?
├─ définit le mot de passe
└─ token SUPPRIMÉ (usage unique)

```

Bonnes pratiques appliquées

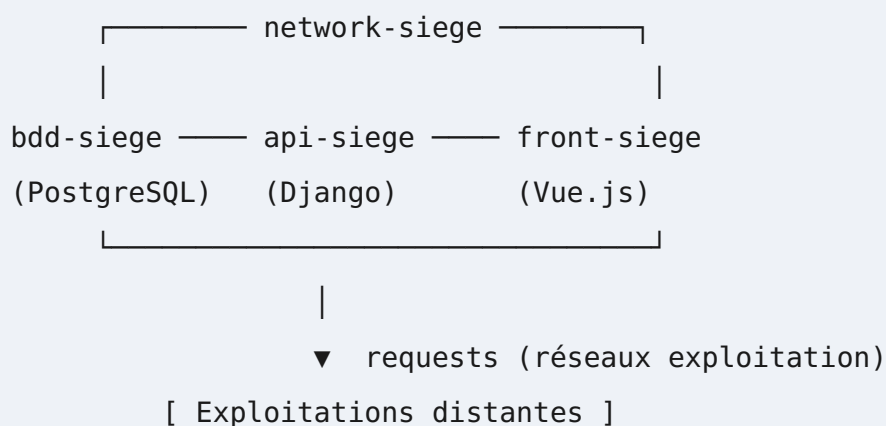
- Secrets (`SECRET_KEY` , `SITE_TOKEN_ENCRYPTION_KEY` , identifiants SMTP) en **variables d'environnement**, jamais commités.
- `DEBUG=False` en production (`core.settings.production`).
- Voir aussi le document transverse `futurekawa-securite.md` à la racine du dépôt.

06 — Configuration & déploiement

Les conteneurs du service

Le backend siège est bien plus simple à déployer que l'exploitation : **pas de Redis, pas de worker, pas de broker** (`docker-compose.siege.yml`).

Conteneur	Image	Rôle
<code>bdd-siege</code>	<code>postgres:16-alpine</code>	Base de données centrale
<code>api-siege</code>	Django	Migrations + serveur API
<code>front-siege</code>	<code>node:20-alpine</code>	Interface Vue.js



Le siège est raccordé aux réseaux des exploitations (`network-exploitation` , `network-exploit-1`) pour pouvoir les appeler.

L'entrypoint

```
# API-siege/entrypoint.sh
1. makemigrations + migrate          # met la base à jour
2. create_default_users              # crée les comptes par défaut
3. runserver 0.0.0.0:5000             # serveur Django
```

Un seul processus, au premier plan — contrairement à l'exploitation qui jongle avec le subscriber MQTT.

Variables d'environnement clés

Variable	Rôle
<code>DJANGO_SETTINGS_MODULE</code>	<code>core.settings.local</code> / <code>production</code>
<code>DJANGO_SECRET_KEY</code>	Clé secrète Django
<code>SITE_TOKEN_ENCRYPTION_KEY</code>	Clé Fernet pour chiffrer les tokens des sites
<code>POSTGRES_*</code>	Connexion base centrale
<code>CORS_ALLOWED_ORIGINS</code>	Origines front autorisées (prod)
<code>EMAIL_HOST_*</code>	SMTP pour les emails d'invitation
<code>FRONTEND_URL</code>	Base des liens d'invitation (<code>/set-password?token=...</code>)

`SITE_TOKEN_ENCRYPTION_KEY` est **critique** : la perdre rend tous les tokens de sites illisibles. À générer avec `Fernet.generate_key()` et à conserver en secret.

Environnements de settings

```
core/settings/
├─ base.py          # commun – INSTALLED_APPS, DRF, chiffrement
├─ local.py         # dev – DEBUG=True, CORS ouvert
├─ production.py    # prod – DEBUG=False, CORS_ALLOWED_ORIGINS
explicite
└─ test.py          # CI – base isolée
```

Démarrage local

```
docker compose -f docker-compose.siege.yml up
```

```
# ou à la main  
cd API-siege  
python manage.py migrate  
python manage.py create_default_users  
python manage.py runserver 0.0.0.0:5000
```

07 — Tests

Approche

Les tests du backend siège utilisent le framework de tests Django (`TestCase`, base jetable) avec les settings `core.settings.test`. Aucune infrastructure externe n'est requise : ni Redis, ni broker, ni exploitation réelle (les appels HTTP sortants sont **mockés**).

Ce qui est couvert

Domaine	Exemples de tests
Authentification	Login, logout, throttle anti brute-force
Invitations	Création utilisateur, set-password, expiration 24 h, usage unique
Sites	CRUD des <code>ExploitationSite</code> , filtrage par pays/statut
Chiffrement	Le token est bien chiffré en base, déchiffré à la lecture
Action <code>sync</code>	Proxy vers <code>/api/all-data/</code> (réponse <code>requests</code> mockée)
Statuts de connexion	<code>active</code> / <code>error</code> / <code>unreachable</code> selon la réponse
Rôles	Un <code>viewer</code> ne peut pas modifier, un <code>admin</code> oui

Mock des appels aux exploitations

Le siège ne doit pas dépendre d'une exploitation en ligne pour ses tests. Les appels `requests.get(...)` sont interceptés :

```
@patch('siege.api.views.http_requests.get')
def test_sync_success(self, mock_get):
```

```
mock_get.return_value.status_code = 200
mock_get.return_value.json.return_value = {...}    # fausse
réponse all-data
# ... POST /api/sites/1/sync/
# → vérifie last_data, last_sync, status == 'active'
```

Lancer les tests

```
cd API-siege
DJANGO_SETTINGS_MODULE=core.settings.test python manage.py test

# ou via le Makefile / la CI
make test-siege
```

Intégration continue

Les tests siège tournent dans la pipeline GitLab CI à chaque push (livrable **CI/CD**). La couverture détaillée figure dans le livrable **Tests** ([04-couverture-api-siege.md](#)).